🏆 AI League #4: Productionizing MCP Servers

Problem Statement: From Localhost to Cloud

📌 Background

League #3 got your Finance MCP servers working locally. You connected them to Cursor over stdio, wrapped Indian financial APIs into tools, and wired up tiered access through Auth0.

Cool. But nobody outside your laptop can use any of it. A server on `localhost` is a prototype. This week, get it off your machine and onto the internet so someone can actually connect, log in, and use it.

🎯 Challenge

Deploy your League #3 MCP server (of any theme) to AWS so that anyone with the URL can connect from Claude Desktop or Claude Web, authenticate, and run theme-specific queries.

- If you didn't build a League #3 server, you can create a new MCP server based on any theme you choose, provided it implements the core League #3 non-functional features like Auth, tiered access/rate limiting, caching, and audit logging (as detailed in "Expected Deliverables").

What "done" looks like: Your MCP server is running on AWS. A user adds the URL in Claude Desktop, gets redirected to Auth0 to log in, and starts querying data via the server's tools. Rate limiting, caching, and audit logging are all working.

Your System Must:

1. **Run on AWS.** Use a single EC2 instance or Lambda, whichever makes more sense for your setup and keeps costs down. Supporting services (cache, database) can run on

AWS or be managed externally.

2. **Run with HTTP.** No localhost demos. Your server needs to be reachable from the public internet using a standard HTTP endpoint on AWS.

3. **Work with Claude.** A user adds your URL in Claude Desktop or Claude Web and it just works.

4. **Have Auth0 authentication working.** Full OAuth 2.1 flow. The user connects, gets redirected to Auth0, logs in, and gets a token. Your server validates that token and shows tools based on the user's tier (Free / Premium / Analyst).

5. **Enforce rate limits by tier.** Free: 30 calls/hr. Premium: 150 calls/hr. Analyst: 500 calls/hr. Per-user, not a blanket cap on the server. When someone goes over their limit, return a `429 Too Many Requests` with a `Retry-After` header.

6. **Cache upstream API responses.** Don't hit NSE or yfinance every time someone asks for the same stock quote. Use TTLs that match how fast the data actually changes. A stock quote goes stale in a minute. Annual financials don't change for months. Also, watch your upstream API quotas.

7. **Log every tool call.** Who called it, their tier, which tool, when, whether it came from cache, how long it took. Logs need to be queryable.

8. **Not crash when things go wrong.** If NSE is having a bad day, serve stale cache if you have it. If you don't, return a clear error. Never send an unhandled exception back to the client.

🖥️ Deployment

Two hosting options on the Kickdrum Playground AWS account:

- **Single EC2 instance** (t2.micro or t2.small) — run your full stack on one box
- **Lambda** — with whatever supporting services you need alongside it

You decide how to structure the services, networking, and everything around it.

Auth0's free tier handles authentication externally. No need to host your own auth server.

⚠️ Constraints & Rules

- Must be accessible over HTTP from the public internet
- Rate limiting must be per-user, not global. You need to tell a Free user apart from an Analyst.
- Upstream API failures should not crash your server or leak raw errors to the client
- Your setup must be reproducible. Another dev should be able to deploy it from your repo by following the README.
- Tag every AWS resource you create. At minimum: `Name`, `Creator`, and `Purpose`. We're sharing the Playground account — untagged resources will be terminated without warning.

🚀 Want to Take It Further?

If you finish the core requirements and want to push further:

Build a chat app that talks to your MCP server. A single-page web app where a user types "How's Reliance doing?", the app sends that to an LLM (OpenAI), the LLM decides which MCP tools to call, your app fires those calls at your MCP server, and the LLM puts together a response from the results.

Think about what that proves. Your MCP server is now serving two different LLM clients. Claude Desktop uses Claude as the brain. Your chat app uses a completely different LLM. Same tools, same auth, same rate limits, same cache. The server doesn't care who's asking.

The app must authenticate through Auth0 (same flow as Claude Desktop) and use at least 3 different tools from your MCP server.

📦 Expected Deliverables

Each team must submit:

1. Working Deployed Server

Running on AWS, accessible over HTTP, connectable from Claude Desktop or Claude Web.

2. Live Demo Showing:

- **Happy Path:** Connect Claude Desktop to your remote server. Authenticate through Auth0. Run a financial query. Show results coming back from AWS.
- **Auth Boundary:** Log in as a Free user. Try a premium-only tool, get a 403. Switch to a Premium user, same tool works.
- **Rate Limiting:** Burn through a Free user's 30-call limit. Show the 429 with the Retry-After header.
- **Caching:** Run the same query twice within the cache TTL. Second call should be faster. Show the audit log marking one as a cache hit.

3. Architecture Diagram

Your AWS setup, how services connect, where auth happens, how requests flow through caching and rate limiting.

4. Technical Explanation

- How you went from stdio to remote transport
- Your Auth0 setup — roles, scopes, JWT validation
- Rate limiting and caching strategy, and why you picked that approach
- What broke during deployment and how you fixed it
- Why you went with EC2 or Lambda and what trade-offs came up
- What your deployed stack costs

5. Repository

README with setup instructions, env variable docs, and everything someone else needs to get your server running.

💡 Tips

1. **Transport first.** Going from stdio to remote HTTP is the biggest jump. Don't touch auth, caching, or rate limiting until your server responds to an HTTP request.

2. **Test locally before deploying.** Full stack on your laptop, Claude Desktop pointed at localhost. Once that works, move to AWS.

3. **Auth0 roles and custom JWT claims work on the free tier.** You don't need to pay for anything.

4. **Audit logs shouldn't slow down responses.** Log asynchronously.

5. **Cache key design matters.** Bad keys mean your cache is either useless or returning wrong data for different queries.
